02b — Multi-Label Transitions

Simultaneous Signal Events

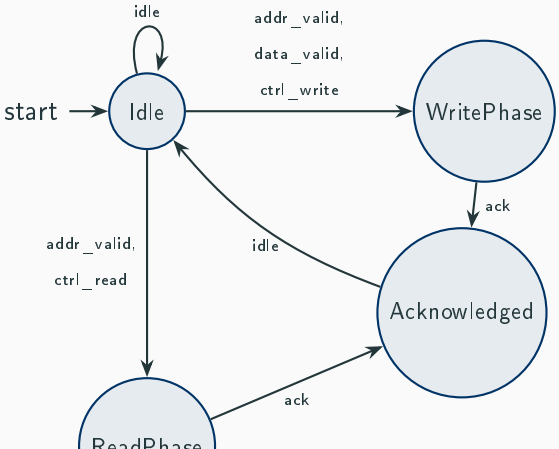
Mariano Cerrutti

Mununu Tutorial

The Concept: Simultaneous Signals

In hardware buses, multiple signals fire in a single clock cycle.

An AXI-like write transaction requires three signals **at once**:



Multi-Label Syntax

```
        }

        transitions {
            // Write transaction: address + data + write control fire together
            transition Idle -> WritePhase on label addr_valid, label data_valid
                , label ctrl_write;
            // Read transaction: address + read control fire together
            transition Idle -> ReadPhase on label addr_valid, label ctrl_read;
            // No transaction
            transition Idle -> Idle on label idle;

            // Slave acknowledges
            transition WritePhase -> Acknowledged on label ack;
            transition ReadPhase -> Acknowledged on label ack;

            // Return to idle after acknowledgment
```

Full Automaton

```
automaton BusController {
    // Master controls address, data, and control signals.
    // ack is uncontrollable (slave-driven acknowledgment).
    controllable {
        label addr_valid;
        label data_valid;
        label ctrl_write;
        label ctrl_read;
        label idle;
    }

    states {
        state Idle initial;
        state WritePhase;
        state ReadPhase;
        state Acknowledged;
    }
```

Single vs. Multi-Label Comparison

	Single-Label (02a)	**Multi-Label** (02b)
Syntax	`on label L`	`on label L1, label L2, ...`
Semantics	One event per transition	Multiple simultaneous events
Use case	Sequential protocols (SPI)	Parallel buses (AXI)
Label count	Exactly one	One or more

Rule of thumb: if your system has signals that must always appear together, model them as a single multi-label transition rather than chaining separate steps.